

Reinforcement Learning

Assignment 1 : Cascading Bandits

Stéphane Meilliez - Etienne Lanzeray

November 2025

1 Part A: Non-contextual Cascading bandit

For the non-contextual Bandit with experimental data we implemented 3 kind of algorithms :

- Cascading UCB
- Cascading klUCB (*optimal for Bernoulli reward*)
- Cascading Thompson Sampling (*uniform prior, Beta posterior*)

Moreover we tried different parameters for the Cascading UCB.

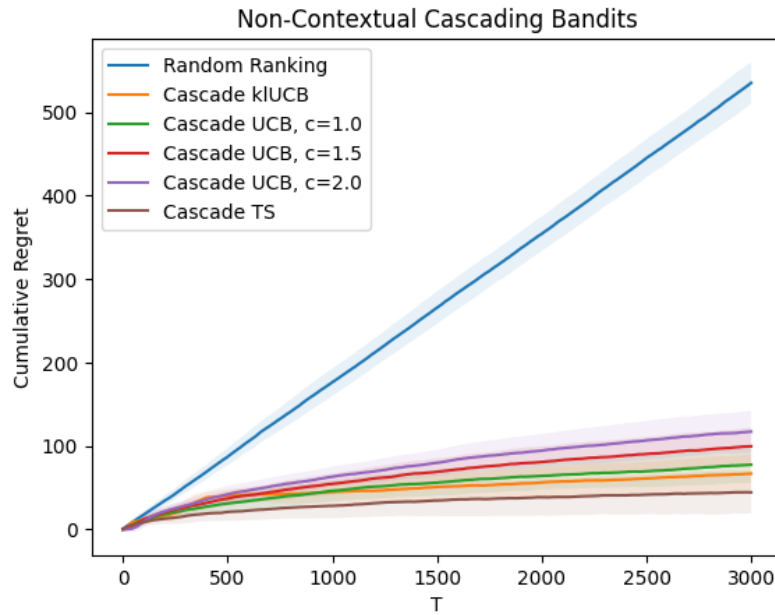


Figure 1: Non- Contextual Cascading Bandits results

As we can see in Fig. 1, klUCB is better than classical UCB but performs worse than Thompson Sampling in finite time.

2 Part B: Contextual Cascading bandit

2.1 The contextual cascading bandit

In the cascading model, for each user the algorithm recommends a ranked list of K items. The user inspects the list from the top and continues downward until encountering an item that satisfies them. If such an item is found, they stop and the reward is 1; otherwise, the user reaches the end of the list with no satisfactory item and the reward is 0.

2.2 Problem setting

We want to apply contextual cascading bandits to a movie-recommendation problem using the MovieLens 100K dataset. The context is represented by user features, while the items correspond to movie features. The environment provides user-movie ratings in a matrix in which nearly 94% of all entries are missing.

At each interaction step, the environment provides a user. Our goal is to *learn which movies to recommend to each user according to the cascade model*.

2.2.1 Modeling User Feedback in the Cascade Setting

For each user–movie pair, the environment must have a click probability that reflects how likely the user is to select the movie when it is shown.

We are provided user/movie ratings, and want to convert this rating into a probability for the user to click on the movie. For that we adopted two approaches.

- linearly map the ratings into probability
- map the ratings into probability with a sigmoid function (*more plausible in our opinion*)

This will, in the end, give us two probability matrices: $\mathcal{M}_{lin}$ & $\mathcal{M}_{sigm}$. The first one gives an environment that should be more suited for a linear bandit, while the second should give a more realistic (but more complex) environment.

Because the dataset rating matrix is highly sparse, we also need to extrapolate the rating a user would give to a movie. We investigate many methods.

1. Naive/Baseline approach:

- If the rating is missing (NaN), we assign a constant default rating (set to 3 in our experiments to represent an uninformative or neutral choice in the absence of data).
- We then apply the rating to probability mapping function to map each rating to a click probability in $(0, 1)$. This provides a simple behavior simulation where higher ratings (4 or 5) correspond to higher click probabilities.

By doing this, while assigning a probability to user–movie pairs with missing ratings we do not take into account any available information. Ideally, we would expect similar users to have similar click probabilities for the same movie.

So we need to take into account the user and movie features. A first baseline would be to give the global (eg all movies) mean probability.

This gives us an RMSE baseline on the training dataset $\{(x_{user,movie}, \text{ProbaMap}(rating))\}$ for each ranking to probability mapping:

$$\text{RMSE}_{Mean}^{lin} = 0.264 \quad \text{RMSE}_{Mean}^{sigm} = 0.273$$

2. Regression Models

We explored several approaches to directly estimate the click probability.

- **Linear Regression:** This provides poor results (not better than the baseline), likely due to the complexity and sparsity of the data:

$$\text{RMSE}_{LinReg}^{lin} = 0.261 \quad \text{RMSE}_{LinReg}^{sigm} = 0.271$$

- **Multi-Layer Perceptron (MLP):** We experimented with several architectures, tuning both the number of layers and the number of neurons. While these models are better than linear regression, their overall performance remained limited:

$$\text{RMSE}_{LinReg}^{lin} = 0.254 \quad \text{RMSE}_{LinReg}^{sigm} = 0.261$$

- **Neural Collaborative Filtering [He+17] and Matrix Factorization**

Neural collaborative filtering learns latent representations (embeddings) for both users ($\mathbf{u}_i$) and movies ($\mathbf{v}_j$). These latent vectors are then mapped to the probability of a user clicking on an item via a function f_θ , which is learned as a multi-layer perceptron:

$$p_{i,j} = f_\theta(\mathbf{u}_i, \mathbf{v}_j).$$

However, this approach suffered from overfitting due to the large number of parameters. To address this issue, we simplified the model by estimating the click probability using the classical inner product of the latent representations:

$$p_{i,j} = \mathbf{u}_i^\top \mathbf{v}_j.$$

This simpler formulation (FixMF) achieved the best performance (see Fig. 2)

$$\text{RMSE}_{FixMF}^{lin} = 0.248 \quad \text{RMSE}_{FixMF}^{sigm} = 0.235$$

In the end¹ this is FixMF we used to extrapolate $\mathcal{M}_{lin}$ et $\mathcal{M}_{sigm}$.

¹We verified that these two matrices gave us a sufficiently realistic environment ie different optimal movie list for different users, and estimated reward for a random list of movie not too high (see notebook section *Predict probability matrices*)

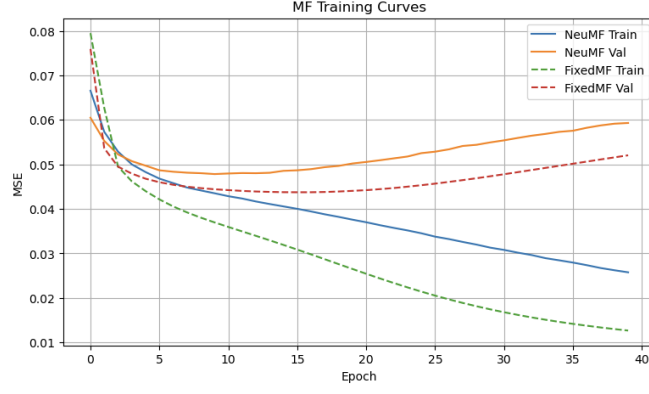


Figure 2: RMSE of different models for interpolating the matrix $\mathcal{M}_{sigm}$.

2.2.2 Feature engineering:

As the bandit algorithms must take both the features of users and the features of movies to modelize the reward, the question of how to combine them arose.

We try different approaches²

- **Concatenation**: a simple method consisting of concatenating the user and movie feature vectors for each pair. This produces 42 combined features but does not account for correlations between features (leading to redundant information).
- **Dimension reduction with PCA**: we ultimately opted for a PCA-based dimensionality reduction applied separately to the user and movie features, followed by concatenation of the reduced representations. (*We kept 15 principal components for both users and movies, capturing more than 95% of the explained variance in each case.*)

2.2.3 The Contextual Cascading Bandit Algorithms

• LinUCB for contextual bandits.

We adapted the classical LinUCB algorithm to the contextual cascading bandit setting.

For each user–movie pair, we construct a feature vector $\mathbf{x}_{eng} \in \mathbb{R}^d$. We assume that the expected reward is given by a linear model:

$$R(\mathbf{x}_{eng}) = \boldsymbol{\theta}^\top \mathbf{x}_{eng},$$

where $\boldsymbol{\theta}$ is a global parameter vector shared across all users and items.

The LinUCB algorithm estimates $\boldsymbol{\theta}$ while balancing exploration and exploitation using confidence bounds on the predicted rewards.

The algorithm maintains:

- a *design matrix* $\mathbf{A} \in \mathbb{R}^{d \times d}$, initialised as $\mathbf{A}_0 = \lambda \mathbf{I}_d$
- a *reward vector* $\mathbf{b} \in \mathbb{R}^d$, initialised as $\mathbf{b}_0 = \mathbf{0}$

and computes the parameter estimate : $\hat{\boldsymbol{\theta}} = \mathbf{A}^{-1}\mathbf{b}$.

Algorithm description:

- Context arrival.** The environment provides a user u_t .
- Feature engineering.** For each movie m_j , we construct the feature vector : $\mathbf{x}_{t,j} = x_{eng}(u_t, m_j)$.
- UCB computation.** For each candidate item:

$$\text{UCB}_{t,j} = \hat{\boldsymbol{\theta}}^\top \mathbf{x}_{t,j} + \alpha \sqrt{\mathbf{x}_{t,j}^\top \mathbf{A}^{-1} \mathbf{x}_{t,j}},$$

where $\alpha > 0$ controls the exploration–exploitation trade-off.

- Recommendation.** We rank movies by UCB score and recommend the top- K items as an ordered list.
- Cascade feedback.** The user examines the list sequentially.
Let V_t be the set of *viewed* items, and let $m_{t,\text{click}}$ be the clicked item if it exists.
Rewards are defined as: $r_{t,j} = \begin{cases} 1 & \text{if } j = m_{t,\text{click}}, \\ 0 & \text{if } j \in V_t \text{ and not clicked.} \end{cases}$
- Parameter update.** We update $\mathbf{A}$ using only the viewed items and $\mathbf{b}$ with the clicked item

$$\mathbf{A} \leftarrow \mathbf{A} + \sum_{j \in V_t} \mathbf{x}_{t,j} \mathbf{x}_{t,j}^\top, \quad \mathbf{b} \leftarrow \mathbf{b} + 1 \times \mathbf{x}_{t,\text{click}}.$$

²We also experimented with the tensor product of user and item features. This approach explicitly models all pairwise interactions and yields richer representations, but at the cost of substantially increased dimensionality and computation time.

Parameter sharing: We maintain a *single* design matrix $\mathbf{A}$ and reward vector $\mathbf{b}$ shared across all users and items. This significantly reduces computational cost and enables information sharing: when a user interacts with a movie, the update benefits the estimation of $\boldsymbol{\theta}$ for all future recommendations.

Using separate matrices per user (or per item) would be memory-intensive, would drastically slow down exploration, and would prevent generalisation across similar users. We rely on the assumption that a single linear model can reasonably approximate user preferences in this context.

- **Linear Thompson Sampling (CascadingLinTS)**

As an alternative to LinUCB, we implemented a Bayesian linear bandit with Thompson Sampling³. We place a Gaussian prior on the global parameter $\boldsymbol{\theta} \sim \mathcal{N}(\mathbf{0}, \lambda^{-1}\mathbf{I})$ and maintain the posterior in closed form through

$$\mathbf{A}_t = \lambda\mathbf{I} + \sum_{s \leq t} \sum_{j \in V_s} \mathbf{x}_{s,j} \mathbf{x}_{s,j}^\top \quad \mathbf{b}_t = \sum_{s \leq t} \mathbf{x}_{s, \text{click}}$$

At each round, we sample $\tilde{\boldsymbol{\theta}}_t \sim \mathcal{N}(\hat{\boldsymbol{\theta}}_t, \sigma^2 \mathbf{A}_t^{-1})$ with $\hat{\boldsymbol{\theta}}_t = \mathbf{A}_t^{-1} \mathbf{b}_t$, rank items by $\mathbf{x}_{t,j}^\top \tilde{\boldsymbol{\theta}}_t$, and recommend the top- K list.

Under cascading feedback, we update $\mathbf{A}_t$ with all viewed items (including non-clicked ones, implicitly reward 0) and update $\mathbf{b}_t$ only with the clicked item (reward 1).

- **Logistic-UCB for contextual cascading bandits (CascadingLogisticUCB1)**

To better match binary click feedback, we also implemented a logistic model where $\mathbb{P}(r_{t,j} = 1 \mid \mathbf{x}_{t,j}) = \sigma(\mathbf{x}_{t,j}^\top \boldsymbol{\theta})$ with $\sigma(z) = \text{sigmoid}(z)$ based on Logistic-UCB-1 [Fau+20].

We fit the regularized MLE

$$\hat{\boldsymbol{\theta}}_t = \arg \max_{\boldsymbol{\theta}} \sum_{s \leq t} \sum_{j \in V_s} \log \text{Bernoulli}(r_{s,j} \mid \sigma(\mathbf{x}_{s,j}^\top \boldsymbol{\theta})) - \frac{\lambda}{2} \|\boldsymbol{\theta}\|_2^2$$

using a Newton method.

For exploration, we compute an optimistic score of the form :

$$\text{UCB}_{t,j} = \sigma(\mathbf{x}_{t,j}^\top \boldsymbol{\theta}_t^{\text{ucb}}) + \beta_t \sqrt{\mathbf{x}_{t,j}^\top \mathbf{A}_t^{-1} \mathbf{x}_{t,j}}$$

where $\mathbf{A}_t = \kappa \lambda \mathbf{I} + \sum_{s \leq t} \sum_{j \in V} \mathbf{x}_{s,j} \mathbf{x}_{s,j}^\top$ and $\boldsymbol{\theta}_t^{\text{ucb}}$ is a projection of the MLE onto an ℓ_2 ball of radius S (as in the article). Finally we rank items by $\text{UCB}_{t,j}$ and recommend the top- K list.

- **Logistic Thompson Sampling with Pólya–Gamma augmentation (PG-TS)**

We implement Thompson Sampling for a logistic reward model following the Pólya–Gamma augmentation scheme [DFE18]. We assume the click probability follows a logistic model:

$$\mathbb{P}(r_{t,j} = 1 \mid \mathbf{x}_{t,j}) = \sigma(\mathbf{x}_{t,j}^\top \boldsymbol{\theta}),$$

and place a Gaussian prior $\boldsymbol{\theta} \sim \mathcal{N}(\mathbf{0}, \lambda^{-1}\mathbf{I})$.

Since the logistic likelihood is not conjugate to the Gaussian prior⁴, we use Pólya–Gamma data augmentation to enable efficient posterior sampling⁵.

Let $\mathcal{D}_t = \{(\mathbf{x}_i, r_i)\}_{i=1}^{n_t}$ denote all observed item-reward pairs up to round t , and let $\mathbf{X}_t$ be the $n_t \times d$ design matrix stacking these contexts. We introduce auxiliary variables $\boldsymbol{\omega} = (\omega_1, \dots, \omega_{n_t})$ where each ω_i follows a Pólya–Gamma distribution. Given $\boldsymbol{\omega}$, the conditional posterior is Gaussian:

$$\boldsymbol{\theta} \mid \boldsymbol{\omega}, \mathcal{D}_t \sim \mathcal{N}(\mathbf{m}_t, \mathbf{V}_t),$$

with covariance matrix and mean given by

$$\mathbf{V}_t = (\mathbf{X}_t^\top \boldsymbol{\Omega}_t \mathbf{X}_t + \lambda \mathbf{I})^{-1}, \quad \mathbf{m}_t = \mathbf{V}_t \mathbf{X}_t^\top \boldsymbol{\kappa}_t,$$

where $\boldsymbol{\Omega}_t = \text{diag}(\omega_1, \dots, \omega_{n_t})$ and $\boldsymbol{\kappa}_t = (r_1 - \frac{1}{2}, \dots, r_{n_t} - \frac{1}{2})^\top$.

At each round t , we perform M Gibbs sampling iterations:

- Sample auxiliary variables: $\omega_i \sim \text{PG}(1, \mathbf{x}_i^\top \boldsymbol{\theta}^{(m-1)})$ for $i = 1, \dots, n_t$
- Sample parameter: $\boldsymbol{\theta}^{(m)} \sim \mathcal{N}(\mathbf{m}_t, \mathbf{V}_t)$ using the updated $\boldsymbol{\Omega}_t$

We then use the final sample $\tilde{\boldsymbol{\theta}}_t = \boldsymbol{\theta}^{(M)}$ to rank items by their predicted click probability $\sigma(\mathbf{x}_{t,j}^\top \tilde{\boldsymbol{\theta}}_t)$ and recommend the top- K list.

Under cascading feedback, the dataset $\mathcal{D}_t$ is updated by adding all viewed items with their observed rewards.

³from Emilie Kaufmann course

⁴A prior distribution is *conjugate* to a likelihood if the posterior belongs to the same distributional family as the prior. For the logistic likelihood $\prod_i \sigma(\mathbf{x}_i^\top \boldsymbol{\theta})^{r_i} (1 - \sigma(\mathbf{x}_i^\top \boldsymbol{\theta}))^{1-r_i}$ combined with a Gaussian prior, the posterior $p(\boldsymbol{\theta} \mid \mathcal{D}) \propto p(\mathcal{D} \mid \boldsymbol{\theta}) p(\boldsymbol{\theta})$ is not Gaussian and has no closed form, making exact posterior sampling intractable.

⁵The key idea is to introduce auxiliary latent variables ω_i with Pólya–Gamma distributions such that the logistic likelihood can be expressed as a Gaussian likelihood conditional on these ω_i . This transforms the intractable posterior into a conditionally Gaussian form, enabling efficient Gibbs sampling by alternating between sampling $\boldsymbol{\omega} \mid \boldsymbol{\theta}$ and $\boldsymbol{\theta} \mid \boldsymbol{\omega}$.

2.3 Experiment run

After implementing the algorithms, we tuned their hyperparameters in order to compare them under their best configurations in $\mathcal{M}_{sigm}$ settings. To do so, each algorithm was run over a grid of candidate parameters and evaluated against a random baseline on a relatively short horizon ($T = 500$).

We then ran the final experiments with $K = 3$ and $T = 1000$. By lack of time, We used only 5 repetitions to reduce variance and obtain more reliable performance estimates.

To keep the total runtime reasonable, we performed all experiments on a subset of 300 movies.

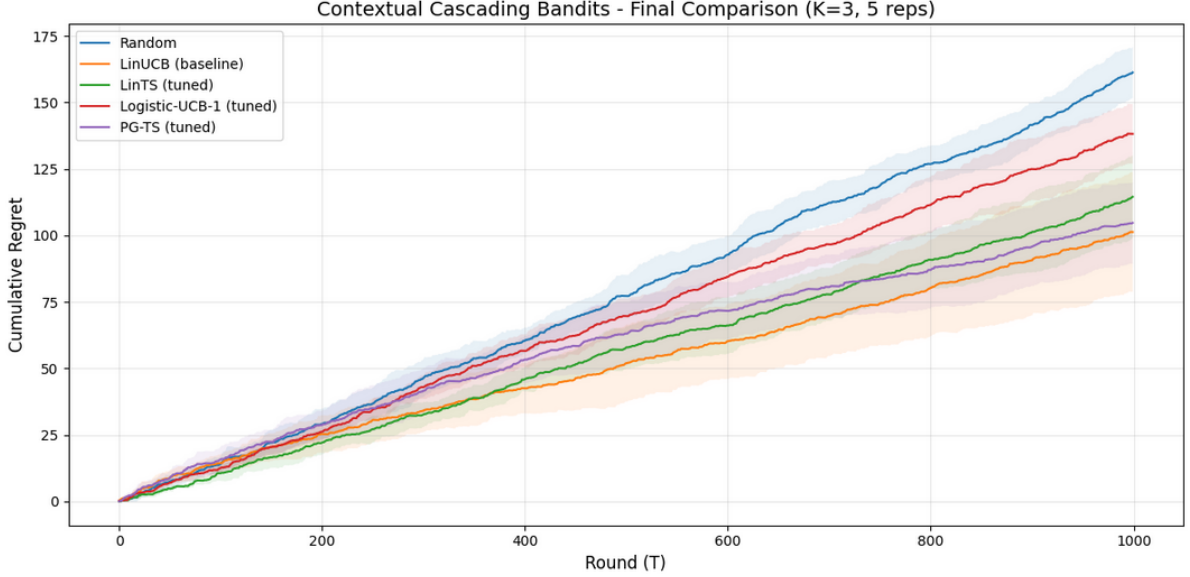


Figure 3: Cumulative regret for contextual algorithms on $\mathcal{M}_{sigm}$ over 1000 rounds (5 repetitions, 300 movies subset). All methods outperform random baseline but show linear growth.

Our algorithms consistently outperform the random baseline. However, we observe linear growth in cumulative regret rather than the sublinear behavior that would be expected under ideal conditions, with no clear plateau emerging by $T=1000$.

Implementation validation. To check our implementation of cascading LinUCB, we use it on synthetic data where click probabilities were generated from a linear model independent of MovieLens ratings. In this controlled setting, we observe the expected sublinear regret, confirming that our algorithmic implementation is correct.

Insufficient exploration horizon. Due to time constraints, experiments were limited to $T=1000$ rounds on a 300-movie subset. This horizon may be insufficient for the algorithms to fully explore and learn optimal recommendations particularly for more complex methods such as LogisticUCB and Thompson Sampling variants. Extending to $T=5000-10000$ would help clarify whether sublinear regret eventually emerges or whether the linear trend persists due to model limitations.

Algorithm comparison. LinUCB consistently outperformed more sophisticated algorithms (LogisticUCB, Thompson Sampling variants) in both environments. This suggests either that the linear assumption is sufficiently accurate in our feature space, or the more complex methods require more extensive hyperparameter tuning.

References

- [DFE18] Bianca Dumitrascu, Karen Feng, and Barbara E. Engelhardt. “PG-TS: Improved Thompson Sampling for Logistic Contextual Bandits”. In: *Advances in Neural Information Processing Systems*. 2018.
- [Fau+20] Louis Faury et al. *Improved Optimistic Algorithms for Logistic Bandits*. 2020. arXiv: 2002.07530 [cs.LG].
- [He+17] Xiangnan He et al. “Neural collaborative filtering”. In: *Proceedings of the 26th international conference on world wide web*. 2017, pp. 173–182.